

Q2 Valid Parentheses

String

Stack

O(n) time · O(n) space

```
function isValid(s) {
  const stack = [];
  const map={'(':')', '[':']', '{':'}';
  for (const c of s) {
    if (!(['('].includes(c)) stack.push(c);
    else if (stack.pop()!==map[c]) return false;
  }
  return stack.length === 0;
}
```

Say: Push openers. For closers, pop + verify match.

Q3 Longest Substring No Repeat

String

Sliding Window

O(n) time · O(1) space

```
function lengthOfLongestSubstring(s) {
  const map = new Map();
  let left = 0, max = 0;
  for (let r = 0; r < s.length; r++) {
    if (map.has(s[r]))
      left = Math.max(left, map.get(s[r])+1);
    map.set(s[r], r);
    max = Math.max(max, r - left + 1);
  }
  return max;
}
```

Say: Sliding window. Jump left past duplicate. Math.max prevents backtrack.

Q4 Find All Anagrams in String

String

Sliding Window

O(n) time · O(1) space

```
function findAnagrams(s, p) {
  if(s.length<p.length) return [];
  const res=[],pC=new Array(26).fill(0),
    sC=new Array(26).fill(0);
  const a=97;
  for(const c of p) pC[c.charCodeAt(0)-a]++;
  for(let i=0;i<s.length;i++){
    sC[s.charCodeAt(i)-a]++;
    if(i>=p.length) sC[s.charCodeAt(i-p.length)-a]--;
    if(sC.join(',')===pC.join(',')) res.push(i-p.length+1);
  }
  return res;
}
```

Say: Fixed window = p.length. Add right, drop left, compare freq arrays.

Q5 Group Anagrams

String

HashMap

O(n·k log k) time · O(n·k) space

```
function groupAnagrams(strs) {
  const map = new Map();
  for (const s of strs) {
    const key = s.split('').sort().join('');
    if (!map.has(key)) map.set(key, []);
    map.get(key).push(s);
  }
  return [...map.values()];
}
```

Say: Sort each string as canonical key. Anagrams share same sorted form.

Q6 Product of Array Except Self

Array

O(n) time · O(1) space

```
function productExceptSelf(nums) {
  const res = new Array(nums.length).fill(1);
  let prefix = 1;
  for(let i=0;i<nums.length;i++){
    res[i]=prefix; prefix*=nums[i];
  }
  let suffix = 1;
  for(let i=nums.length-1;i>=0;i--){
    res[i]*=suffix; suffix*=nums[i];
  }
  return res;
}
```

Say: Two passes — prefix left to right, suffix right to left. No division.

Q7 Maximum Subarray — Kadane's

Array

Greedy

O(n) time · O(1) space

```
function maxSubArray(nums) {
  let curr = nums[0], max = nums[0];
  for (let i = 1; i < nums.length; i++) {
    curr = Math.max(nums[i], curr + nums[i]);
    max = Math.max(max, curr);
  }
  return max;
}
// [-2,1,-3,4,-1,2,1,-5,4] -> 6
```

Say: Extend or restart? curr = max(num, curr+num). Greedy single pass.

Q8 Autocomplete — Subsequence

String

Two Pointers

O(n·m) time · O(1) space

```
function isSubsequence(name, search) {
  let i = 0;
  for(let j=0; j<name.length && i<search.length; j++){
    if(name[j]===search[i]) i++;
  }
  return i === search.length;
}
function autocomplete(names, query) {
  return names.filter(n => isSubsequence(n, query));
}
```

Say: Two pointers. Advance search pointer only on match. Exact Karat Q.

Q9 Implement Debounce

Closure

Design

O(1) time · O(1) space

```
function debounce(fn, delay) {
  let timer;
  return function(...args) {
    clearTimeout(timer);
    timer = setTimeout(() => {
      fn.apply(this, args);
    }, delay);
  };
}
// Usage:
const s = debounce(fetchAPI, 300);
input.addEventListener('input', e=>s(e.target.value));
```

Say: Closure holds timer. Each call clears+resets. fn fires only after silence.

Q10 Top K Frequent Elements

Array

HashMap

O(n log n) time · O(n) space

```
function topKFrequent(nums, k) {
  const map = new Map();
  for(const n of nums)
    map.set(n, (map.get(n)||0)+1);
  return [...map.entries()]
    .sort((a,b) => b[1]-a[1])
    .slice(0,k)
    .map(e => e[0]);
}
// Bonus: bucket sort = O(n)
```

Say: Count frequencies → sort desc → take k. Mention bucket sort = O(n).